Architecture — Deep Dive

> 6 Spring Boot microservices · Kafka saga · Kong gateway · Redis · Postgres-per-service · Floci (AWS sim) How everything ships end-to-end → WORKFLOW.md

---

1. System Overview

Every request enters through Kong (rate-limited per IP). Services never call each other over REST — all cross-service communication is Kafka events.

```
flowchart TB
    C[Client] --> KONG[Kong Gateway :8000<br/>rate-limit · routing · WAF]
    KONG --> U[user-service :8081]
    KONG --> P[product-service :8082]
    KONG --> O[order-service :8083]
    KONG --> I[inventory-service :8084]
    KONG --> PAY[payment-service :8085]

    U --- UDB[(userdb)]
    P --- PDB[(productdb)]
    O --- ODB[(orderdb)]
    I --- IDB[(inventorydb)]
    PAY --- PAYDB[(paymentdb)]

    U -.sessions.- R[(Redis)]
    P -.cache 15m TTL.- R
    I -.reservation tracking.- R

    O <-->|events| K[(Kafka)]
    I <-->|events| K
    PAY <-->|events| K
    P -->|events| K
    K --> N[notification-service :8086]
    N --> SES[Floci SES email]
```

- **DB-per-service** — no shared tables; each service owns its schema (Flyway migrations run on startup).
- **Redis, 3 distinct jobs** — JWT session store (logout, enforced at user-service only), product cache, and per-order reservation tracking for inventory. The oversell guard is *not* the Redis lock it used to be: that lock was released in a `finally` block while the method was still `@Transactional`, so it was gone before the write committed and 40 concurrent orders sold 18 units out of 10. It is now a single conditional `UPDATE ... WHERE quantity - reserved >= :qty`.

2. Order Saga (choreography — no orchestrator)

Each service reacts to events and emits the next one. Failure at any step emits a compensating event.

```
sequenceDiagram
    autonumber
    participant C as Client
    participant O as order-service
    participant K as Kafka
    participant I as inventory-service
    participant P as payment-service
    participant N as notification-service

    C->>O: POST /orders
    O->>O: save order (PENDING)
    O->>O: COMMIT
    O->>K: order.created (after commit — see note)
    K->>I: order.created
    I->>I: atomic conditional UPDATE → reserve stock
    I->>K: inventory.updated (ok/fail)
    K->>O: inventory.updated
    K->>P: inventory.updated
    alt stock reserved
        P->>K: payment.processed (ok/fail)
        K->>O: payment.processed → PAID / CANCELLED
        K->>I: payment.processed → deduct or release stock
    else out of stock
        O->>O: mark CANCELLED
        O->>K: order.cancelled (compensation)
        K->>I: order.cancelled → release stock
    end
    K->>N: order.created / payment.processed / order.cancelled
    N->>C: email via SES
```

5 topics: `order.created` · `inventory.updated` · `payment.processed` · `order.cancelled` · `product.created` One consumer group per service; 3–6 concurrent listeners per topic for throughput.

Why the diagram shows COMMIT before the publish. order-service used to send `order.created` from inside the `@Transactional` method. The Kafka send goes out immediately; the commit happens when the method returns. Under a 20-order burst inventory-service replied within milliseconds — before the order row was visible to anyone else — and order-service's reply handler looked the order up, found nothing, and returned without a word. Measured on a live cluster: 21 orders, 21 `order.created`, 21 `inventory.updated` replies, consumer lag 0, and **4 orders stranded at PENDING with no log line at all**. Events now go out on `AFTER_COMMIT`, so a reply cannot arrive before the order exists.

If the process dies between the commit and the send, the event is still lost — that residual gap is covered by a stale-order reaper rather than by a transactional outbox, so a lost event degrades to a cancelled order rather than a stuck one.

---

3. Auth Flow (JWT per service, Redis sessions at user-service)

```
sequenceDiagram
    participant C as Client
    participant U as user-service
    participant R as Redis
    participant S as any other service

    C->>U: POST /login
    U->>R: store session
    U-->>C: JWT (sub, userId, role)
    C->>S: request + JWT
    S->>S: verify signature with the shared secret
    alt invalid or missing
        S-->>C: 403
    else valid
        S-->>C: 200
    end
    C->>U: request + JWT
    U->>R: session still present?
    alt logged out
        U-->>C: 403
    else valid
        U-->>C: 200
    end
```

Each service verifies the signature itself rather than trusting the gateway — Kong is not the only route to a pod, since `kubectl port-forward` goes straight past it. The token carries a **userId** claim, which is what lets order-service identify the caller without reading a user id out of the request body.

Two things this diagram deliberately shows, because both used to be described wrongly here:

1. Until the authorisation work, five of the six services had no security configuration at all. `POST /api/orders` with no `Authorization` header returned **200**. An earlier version of this diagram showed *every* service asking Redis whether the session was valid; no service except user-service ever did that.
2. **Logout is still only enforced at user-service.** It deletes the Redis session, so user-service rejects the token immediately — but the other five verify the signature and expiry only, and will keep accepting that token until it expires (24h). Closing that needs either a shared revocation check or short-lived tokens with refresh. It is a known gap, not a solved problem.

---

4. Cloud Topology (Floci EKS)

```
flowchart LR
    NET[Internet] --> ALB[ALB :443<br/>SSL + WAF<br/>public subnet]
    subgraph EKS[EKS cluster — private subnets]
        ALB --> KONG[Kong]
        KONG --> SVC[6 service Deployments + HPA]
    end
    subgraph DATA[private subnets — see note]
        PG[(Postgres ×5)]
        RD[(Redis)]
        KF[(Kafka)]
    end
    SVC --> PG & RD & KF
    ECR[(ECR — image per service,<br/>tag = git SHA)] --> SVC
```

- **This is a 2-tier deployment, not 3.** `scripts/01-vpc.sh` creates data subnets and a `db-sg`, but nothing uses them: Postgres, Redis and Kafka are StatefulSets, so they run as pods on the EKS nodes in the **private** subnets, and `db-sg` is never attached to anything. Isolation is enforced in Kubernetes instead — see `k8s/security/networkpolicy.yaml`, which is *stricter* than `db-sg` would have been: `db-sg` let any pod carrying `eks-sg` reach any database on 5432, whereas the policies allow only order-service → postgres-order, and so on. Security groups filter per-ENI and cannot express per-pod rules.
- 2 EKS node groups: `general` (m5.large — most services, Kong) and `high-mem` (r5.large — Kafka consumers).
- Manifests in k8s/: `namespace/ postgres/ redis/ kafka/ kong/ services/ monitoring/`. Same manifests work on real AWS — only env vars change.

---

5. CI/CD (summary — full detail in WORKFLOW.md)

```
flowchart LR
    PR[push / PR] --> T[test ×6 parallel matrix]
    T --> B[mvn -T 1C build + docker build ×6]
    B --> S[Trivy scan]
    S --> K[k3d cluster → deploy all manifests]
    K --> SM[saga smoke test → order PAID]
    SM --> G{{main only: human approval}} --> PROD[deploy-prod]
```

---

6. Per-Service Detail

Service	Port	Owns	Publishes	Consumes	Special
user	8081	users, auth	—	—	JWT + Redis sessions
product	8082	catalog	`product.created`	—	Redis cache 15-min TTL
order	8083	orders, saga state	`order.created`, `order.cancelled`	`inventory.updated`, `payment.processed`	Saga initiator
inventory	8084	stock	`inventory.updated`	`order.created`, `order.cancelled`, `payment.processed`, `product.created`	Atomic conditional UPDATE — no oversell
payment	8085	payments	`payment.processed`	`inventory.updated`	Pays only after stock reserved
notification	8086	— (stateless)	—	`order.created`, `payment.processed`, `order.cancelled`	SES email